Rapport de PRE

Prédiction Boursière Optimisation de Portefeuille Multi-Objectifs (MOSMA)

BOURREE Jean-Baptiste

Sous la direction de : *Bouh Adhe*

20/08/2026

Remerciements

Je souhaite tout d'abord remercier Bouh Adhe, ma maître de stage, sans qui ce stage n'aurait pas été possible. Je souhaite ensuite remercier Rei, son assistant chercheur, qui a pu m'aider et me conseiller sur de nombreuses problématiques. Ensuite, je souhaite remercier l'IRO (International Relation Office) qui m'a permis d'obtenir et d'étendre mon visa. Enfin, je souhaite remercier l'ENSTA qui a apporté une structure à ce stage.

Résumé

Ce document présente une étude exhaustive de l'optimisation de portefeuille appliquée au marché boursier indonésien (IDX), avec pour finalité la conception et le déploiement d'une plateforme web interactive destinée au département P2MS de l'ITB. Face aux limites des approches continues classiques comme celle de Markowitz, nous explorons des méthodes capables de respecter des contraintes financières réelles et strictes, telles que l'allocation discrète (achat par lots entiers) et la prise en compte asymétrique des coûts de transaction. Le rapport met en lumière l'implémentation d'une stratégie hybride dynamique basée sur la détection de régime de marché, puis détaille le développement d'un puissant algorithme métaheuristique multi-objectifs (MOSMA) couplé à la méthode de tri NSGA-III. En parallèle, des tentatives de prédiction des séries temporelles via des réseaux de neurones profonds (LSTM) sont menées et décryptées. Ce manuscrit est structuré de manière à expliciter les fondements mathématiques de ces algorithmes, l'architecture logicielle développée, et propose une évaluation critique des performances issues des backtests hors-échantillon, en soulignant l'impact décisif du biais du survivant.

Mots-clés : Optimisation de Portefeuille, Algorithme Génétique, MOSMA, NSGA-III, Backtesting, Biais du Survivant, Finance Quantitative, Apprentissage Automatique, LSTM,.

Table des matières

Remerciements **1**

Résumé **2**

Table des matières **3**

Table des figures **5**

1 Introduction Générale **6**
1.1 Contexte de l'étude 6
1.2 Problématique 6
1.3 Objectifs du rapport 6

2 État de l'Art et Fondements Théoriques **8**
2.1 Théorie Moderne du Portefeuille (Markowitz) et ses défauts 8
2.1.1 Le problème de l'allocation continue vs discrète 8
2.1.2 L'inertie temporelle du modèle 8
2.2 MOSMA et Algorithmes Métaheuristiques 9
2.2.1 Vitesse de Convergence et Complexité Temporelle 10
2.3 Différence entre NSGA-II et NSGA-III 10
2.4 Réseaux de Neurones LSTM (Long Short-Term Memory) 12

3 Méthodologie et Préparation des Données **14**
3.1 Sélection des Actifs et Période d'Étude 14
3.2 Fonction de Coût Transactionnel 14

4 Approche Classique et Évaluation Dynamique **15**
4.1 Création de la Frontière Efficiente (CVXPY) 15
4.2 Premier Backtest : Évaluation Continue 16

5 Conception d'un Algorithme Hybride **18**
5.0.1 Fonctionnement Mathématique 18

6 De l'Algorithme Génétique à MOSMA **20**
6.1 Nécessité de l'optimisation discrète et Algorithme Génétique 20
6.2 Implémentation algorithmique de MOSMA (Logique NSGA-II) 21
6.3 Uniformisation via NSGA-III 22

7 Création de la Plateforme Web (Streamlit) **24**
7.1 Paramétrage de l'Investissement 24
7.2 Visualisation et Sélection sur le Front de Pareto 25
7.3 Allocation Hybride et Détection de Régime 26
7.4 Analyse Comparative par Backtest (Validation Out-of-Sample) 27

8 Tentative de Prédiction Boursière via LSTM **28**
8.1 Préparation des données et Modélisation 28
8.2 Analyse de la Prédiction à 1 Jour (Fenêtre Glissante Réelle) 29
8.3 Analyse de la Prédiction Récursive à 7 Jours 29
8.4 Conclusion sur l'utilisation du LSTM 30

9 Conclusion et Perspectives **31**
9.1 Bilan de l'Étude 31
9.2 Le Biais du Survivant : Une Limite Majeure 31
9.3 Perspectives Futures 32

Bibliographie **33**

A Codes Sources Python Développés **34**
A.1 Prédiction LSTM (`ml_stage.py`) 34
A.2 Application Streamlit (`app.py`) 35
A.3 Moteur de Backtest (`backtest_engine.py`) 36
A.4 Stratégies Multi-Objectifs (`strategies.py`) 37
A.5 Fonctions Utilitaires (`utils.py`) 39

Table des figures

2.1 NSGA-III 11

4.1 L'univers des portefeuilles possibles et la Frontière Efficiente discrète. . . . 16
4.2 Évolution de la valeur du capital : Markowitz (Best Sharpe Ratio) vs Benchmark (1/N). 17

5.1 Évolution des portefeuilles et détection des krachs avec l'Algorithme Hybride. 19

6.1 Frontière générée par l'Algorithme Génétique : un amas de points irrégulier et sous-optimal. 21
6.2 Frontière générée par MOSMA (logique NSGA-II) avec une nette amélioration de la convergence. 22
6.3 Uniformisation absolue du Front de Pareto grâce au filtre NSGA-III. 23

7.1 Paramètres financiers globaux. 25
7.2 Sélection des actifs et portefeuille actuel. 25
7.3 Interface du Front de Pareto interactif et du plan d'exécution ciblé. 26
7.4 Tableau de l'Allocation Hybride avec indicateur de Régime de Marché (Alpha). 26
7.5 Évolution historique des différentes stratégies et métriques de risque final. . 27

8.1 Prédiction jour par jour de ADRO.JK sur 60 jours. 29
8.2 Prédiction Autorégressive (Récursive) de ADRO.JK sur 7 jours. 30

Chapitre 1

Introduction Générale

1.1 Contexte de l'étude

Les marchés financiers modernes, tels que la Bourse d'Indonésie (IDX) étudiée dans le cadre de ce projet, sont caractérisés par une volatilité importante et un bruit stochastique complexe.La prévision est l'allocation des différents actifs financiers représentent donc un défi majeur de l'industrie financière. Le but final de cette étude est de créer une application web qui sera hébergée à la fin du stage sur le site du P2MS (Département de mathématiques appliquées à la finance de l'Institut Technologique de Bandung - ITB). Ce site permettra à tout utilisateur, d'optimiser un portefeuille d'actions en fonction de paramètres personnalisés (budget, frais de courtage, contraintes de poids), en s'appuyant sur des méthodes d'optimisation.

1.2 Problématique

La conception d'un outil d'optimisation de portefeuille pour le marché indonésien soulève un défi algorithmique et mathématique majeur : comment concevoir et implémenter une plateforme web accessible qui permette d'optimiser un portefeuille d'actions en intégrant des contraintes réelles et strictes, telles que l'obligation légale d'achat par lots entiers et la prise en compte des coûts de transaction ?

1.3 Objectifs du rapport

Ce mémoire vise à :

— Concevoir et déployer une application web interactive accessible à tous, hébergée par le département de mathématiques appliquées (P2MS) de l'ITB.

— Intégrer des contraintes spécifiques au marché indonésien, notamment l'obligation mathématique et technique d'acheter et de vendre des actions exclusivement par

lots de 100 unités.

— Développer des approches d'optimisation de portefeuille hybrides et avancées (algorithmes génétiques, MOSMA) pour pallier les défauts des modèles de Markowitz.
— Mettre en place un simulateur de *Rolling Window Backtest* pour évaluer et comparer avec précision les performances des stratégies hors échantillon.
— Implémenter des algorithmes de *Machine Learning* (LSTM multivarié) pour tenter de prédire la valeur des actifs à une date ultérieure

Chapitre 2

État de l'Art et Fondements Théoriques

2.1 Théorie Moderne du Portefeuille (Markowitz) et ses défauts

L'optimisation moyenne-variance de Markowitz cherche à minimiser la variance du portefeuille pour un niveau de rendement donné. Le problème se pose ainsi :

$$\min_{w} w^T \Sigma w \tag{2.1}$$

Sous les contraintes :

$$w^T \mu \geq \mu_p, \quad \sum_{i=1}^{N} w_i = 1, \quad w_i \geq 0 \tag{2.2}$$

où w est le vecteur des poids, Σ la matrice de covariance, μ le vecteur des rendements espérés, et μ_p le rendement cible.

2.1.1 Le problème de l'allocation continue vs discrète

Bien que cette théorie offre un cadre rigoureux, elle présente un défaut lors de son application sur la bourse indonésienne (IDX). L'optimisation produit des poids continues. Or, sur l'IDX, les actions ne s'achètent pas à l'unité ni en fraction, elles s'achètent par lots (1 lot = 100 actions). Un simple arrondi des pondérations pour s'adapter à des nombres entiers conduit à des portefeuilles sous-optimaux. Il faut donc utiliser des algorithmes qui travaillent sur un espace discret.

2.1.2 L'inertie temporelle du modèle

Un second défaut du modèle de Markowitz est son incapacité à mettre plus d'importance sur les données récentes que les anciennes. Le modèle se base sur des rendements historiques moyens μ calculés sur une période globale. Si une action a généré des profits co-

lossaux il y a plusieurs mois mais se trouve actuellement dans une forte tendance baissière, sa moyenne historique restera élevée. Le solveur de Markowitz cherchera alors à investir massivement dans cet actif en déclin, ignorant la dynamique de marché actuelle. C'est pourquoi des approches prédictives ou l'utilisation de momentum doivent être superposés au modèle.

2.2 MOSMA et Algorithmes Métaheuristiques

L'algorithme MOSMA (Multi-Objective Slime Mould Algorithm) s'appuie sur les principes du Slime Mould Algorithm (SMA). Le SMA est une méthode d'optimisation stochastique novatrice basée sur le mode d'oscillation du myxomycète (*slime mould*) dans la nature. Ce modèle mathématique utilise des poids adaptatifs pour simuler le processus de rétroaction positive et négative de l'onde de propagation du myxomycète, générée par un oscillateur biologique. Cette mécanique permet de former le chemin optimal pour relier les sources de nourriture, démontrant ainsi une excellente capacité d'exploration et une forte propension à l'exploitation.

Le modèle simule spécifiquement les changements morphologiques et le comportement de recherche de nourriture de l'organisme unicellulaire Physarum polycephalum. Lorsqu'une veine s'approche d'une source de nourriture, l'oscillateur biologique produit une onde de propagation qui augmente le flux cytoplasmique à travers la veine ; plus le cytoplasme coule rapidement, plus la veine s'épaissit[cite : 10]. Grâce à cette combinaison de rétroactions positives et négatives, le myxomycète établit le chemin optimal pour se connecter aux nutriments de manière supérieure.

De plus, l'organisme est capable d'ajuster dynamiquement ses modèles de recherche en fonction de la qualité de la source de nourriture[cite : 10]. Si la concentration en nourriture est élevée, il utilise une méthode de recherche confinée à cette région spécifique, concentrant son exploration sur la ressource identifiée. À l'inverse, si la qualité initiale de la nourriture est faible, il quitte la zone pour explorer d'autres sources alternatives dans l'environnement.

Sur le plan de la modélisation mathématique, le comportement biologique du SMA est formalisé par des équations régissant la mise à jour de la position des individus (les veines du myxomycète). La position $\vec{X}$ d'un individu à l'itération $t+1$ est mise à jour selon la règle globale :

$$\vec{X}(t+1) = \begin{cases} \text{rand} \cdot (UB - LB) + LB, & \text{si rand} < z \\ \vec{X}_b(t) + \vec{vb} \cdot (\vec{W} \cdot \vec{X}_A(t) - \vec{X}_B(t)), & \text{si } r < p \\ \vec{vc} \cdot \vec{X}(t), & \text{si } r \geq p \end{cases} \tag{2.3}$$

où $\vec{X}_b$ est la meilleure position trouvée, $\vec{X}_A$ et $\vec{X}_B$ sont deux individus choisis aléatoire-

ment, $\vec{vb}$ et $\vec{vc}$ sont des paramètres d'oscillation, $\vec{W}$ est le poids adaptatif, et UB et LB définissent les limites de l'espace de recherche[cite : 10].

La variable p contrôle la probabilité de transition entre l'exploration de nouvelles zones et l'exploitation d'une ressource connue, et est définie par :

$$p = \tanh |S(i) - DF| \tag{2.4}$$

où $S(i)$ est l'évaluation (fitness) de la position actuelle et DF est la meilleure évaluation trouvée globalement.

Le SMA utilise le poids adaptatif $\vec{W}$ pour simuler la fréquence d'oscillation à différentes concentrations de nourriture. La formulation mathématique de ce poids est la suivante :

$$\vec{W}(i) = \begin{cases} 1 + r \cdot \log\left(\frac{bF - S(i)}{bF - wF} + 1\right), & \text{si } S(i) \text{ est dans la meilleure moitié} \\ 1 - r \cdot \log\left(\frac{bF - S(i)}{bF - wF} + 1\right), & \text{sinon} \end{cases} \tag{2.5}$$

où bF (best fitness) et wF (worst fitness) sont respectivement la meilleure et la pire évaluation de l'itération courante, et $r \in [0, 1]$ simule l'incertitude biologique de la contraction veineuse[cite : 10]. Cette faculté à maintenir un équilibre adaptatif mathématique entre la phase d'exploration (gérée par $\vec{vc}$) et la phase d'exploitation (gérée par $\vec{vb}$ et $\vec{W}$) rend cet algorithme stochastique particulièrement puissant pour résoudre les problèmes sous contraintes discrètes.

2.2.1 Vitesse de Convergence et Complexité Temporelle

L'oscillation rapide dans les premières phases de recherche permet au SMA une exploration vaste et une convergence accélérée, tandis que l'oscillation plus faible dans les phases finales garantit une recherche précise près de la solution optimale. Bien que la courbe d'évaluation (fitness) puisse présenter des oscillations locales, sa valeur globale diminue continuellement. Il convient toutefois de noter que les évaluations mathématiques liées aux facteurs d'oscillation exigent une puissance de calcul légèrement supérieure à celle de métaheuristiques plus rudimentaires, augmentant marginalement le temps d'exécution (wall-clock time) par itération, mais ce coût est largement compensé par la réduction drastique du nombre d'itérations nécessaires pour converger.

2.3 Différence entre NSGA-II et NSGA-III

Pour résoudre des problèmes d'optimisation à plusieurs objectifs (comme maximiser le rendement tout en minimisant le risque), les algorithmes génétiques de tri non dominé (NSGA) sont particulièrement reconnus. Il est crucial de distinguer NSGA-II et NSGA-III.

NSGA-II (utilisé comme base de nombreux algorithmes comme MOSMA) s'appuie sur deux concepts clés : le tri non dominé pour classer les solutions par fronts de Pareto, et la distance de congestion (Crowding Distance). La distance de congestion estime la densité des solutions autour d'un point donné dans l'espace des objectifs. Mathématiquement, elle se calcule comme la somme des distances normalisées entre les voisins immédiats d'une solution pour chaque objectif[cite : 16]. Cela incite l'algorithme à privilégier les solutions situées dans des zones moins peuplées, favorisant ainsi la diversité du front de Pareto.

Cependant, NSGA-II peine à maintenir une distribution parfaitement uniforme lorsque la complexité du problème augmente ou lorsque le nombre d'objectifs dépasse deux (comme nous pourrons l'observer plus tard). C'est ici qu'intervient NSGA-III. Plutôt que d'utiliser la distance de congestion, NSGA-III utilise des points de référence (Reference Directions) prédéfinis de manière équidistante sur un plan.

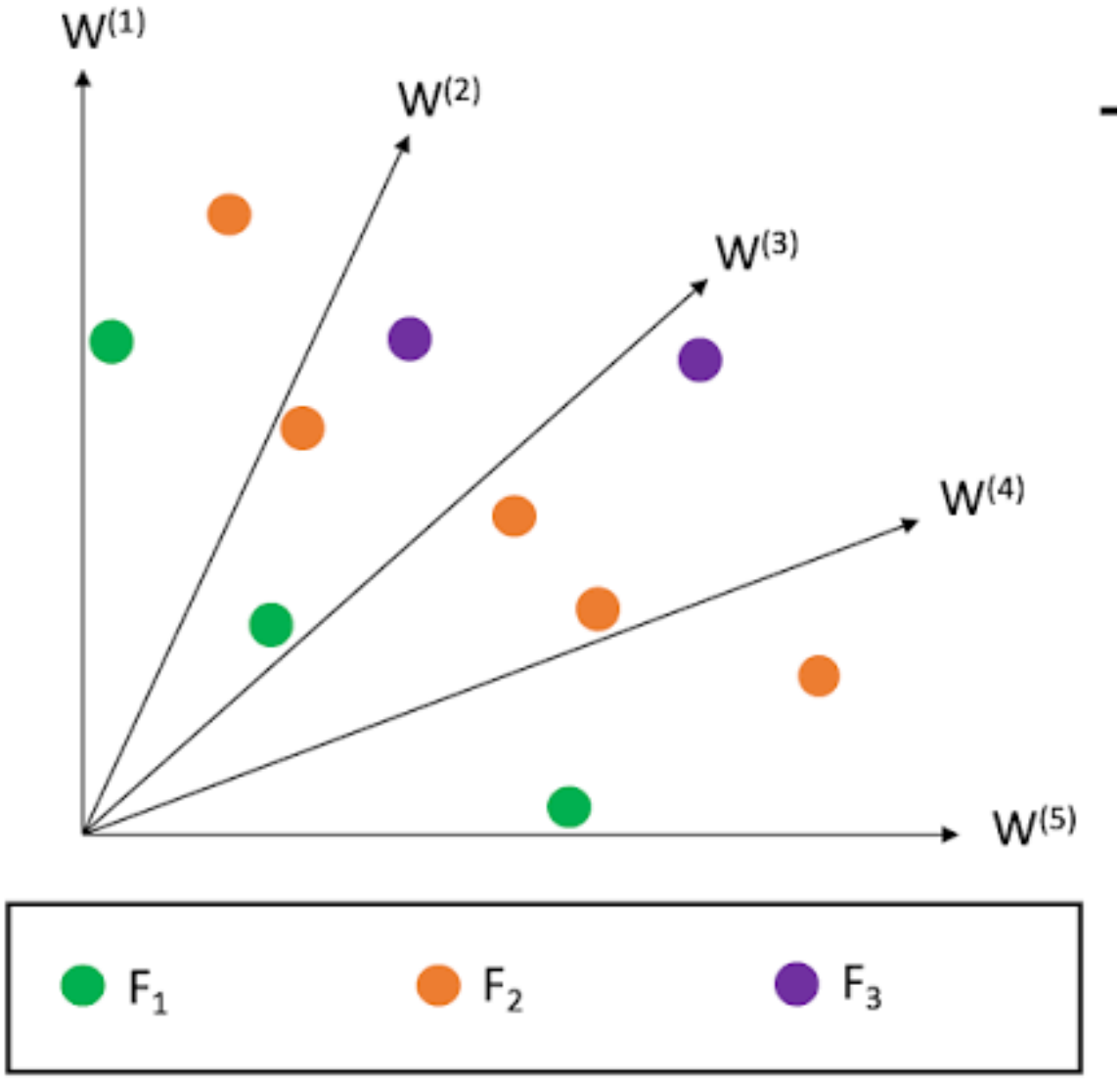


FIGURE 2.1 – NSGA-III

L'algorithme associe chaque membre de la population au point de référence dont il est le plus proche (via une distance perpendiculaire). Ce mécanisme de « niching » garantit que le front de Pareto final aura des points qui sont repartis de manière homogène.

2.4 Réseaux de Neurones LSTM (Long Short-Term Memory)

Pour anticiper les tendances futures du marché boursier et non pas seulement réagir au passé, notre modèle d'optimisation a besoin de prédictions robustes. Pour cela, nous allons tenter d'utiliser un reseau de neurones, Le Long Short-Term Memory (LSTM).

Soit x_t le vecteur des caractéristiques d'entrée au pas de temps t (prix, volume, indicateurs techniques), h_{t-1} l'état caché (la sortie) du pas de temps précédent, et C_{t-1} l'état de la cellule précédent. Les matrices de poids W et les vecteurs de biais b sont les paramètres appris par le modèle. La mécanique complète d'une cellule LSTM repose sur les étapes et équations suivantes :

1. La Porte d'Oubli (Forget Gate) f_t

Cette porte détermine quelle proportion de la mémoire passée C_{t-1} doit être effacée ou conservée. Elle utilise une fonction d'activation sigmoïde (σ) qui compresse les valeurs entre 0 (oubli total) et 1 (conservation totale).

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{2.6}$$

2. La Porte d'Entrée (Input Gate) i_t et l'état candidat $\tilde{C}_t$

L'algorithme doit ensuite décider quelles nouvelles informations stocker dans la cellule de mémoire. L'Input Gate i_t définit les valeurs à mettre à jour, tandis qu'une couche avec une activation tangente hyperbolique (tanh) génère un vecteur de nouveaux candidats $\tilde{C}_t$ (valeurs entre -1 et 1).

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{2.7}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{2.8}$$

3. Mise à jour de l'État de la Cellule C_t

L'ancien état C_{t-1} est mis à jour vers le nouvel état C_t. On multiplie l'ancien état par le vecteur d'oubli f_t, et on y additionne les nouvelles informations pondérées par la porte d'entrée. L'opérateur $\odot$ représente le produit de Hadamard (multiplication élément par élément).

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{2.9}$$

4. La Porte de Sortie (Output Gate) o_t et l'État Caché h_t

Enfin, le réseau décide de la valeur de sa prédiction cachée pour le pas de temps t. La

porte de sortie détermine quelle partie de l'état de la cellule sera exposée, et cet état est filtré par une fonction tanh pour stabiliser le réseau.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{2.10}$$

$$h_t = o_t \odot \tanh(C_t) \tag{2.11}$$

Dans notre implémentation Python, cette machinerie lourde est encapsulée dans la couche `LSTM()`. Le modèle traite des séquences glissantes de 30 jours pour prédire la variation du 31ème jour.

Chapitre 3

Méthodologie et Préparation des Données

3.1 Sélection des Actifs et Période d'Étude

Dans un premier temps, afin d'assurer la stabilité et la pertinence de nos modèles, l'étude s'est concentrée exclusivement sur les 30 premiers stocks indonésiens (l'indice IDX30). L'architecture du site web a été conçue pour permettre à l'utilisateur final d'ajouter manuellement n'importe quel actif de son choix (ex : NVDA, TSLA) pour personnaliser son portefeuille. Toutefois, le cœur du moteur algorithmique est configuré pour le marché indonésien, imposant un système transactionnel strict où tous les achats et ventes doivent se faire par blocs indivisibles de 100 actions (1 lot).

3.2 Fonction de Coût Transactionnel

Le rééquilibrage du portefeuille et l'achat initial ne sont pas gratuits. Sur le marché indonésien, l'achat et la vente possèdent un coût asymétrique (la vente étant souvent plus taxée). Ces frais sont modélisés par les coefficients $\alpha_{buy} = 0.15\%$ et $\alpha_{sell} = 0.25\%$. La fonction de coût total transactionnel (TC) s'écrit de la manière suivante :

$$TC = \alpha_{buy} \sum_{i=1}^{N} (k_i^{cible} - k_i^{prev})^+ \cdot P_i + \alpha_{sell} \sum_{i=1}^{N} (k_i^{prev} - k_i^{cible})^+ \cdot P_i \tag{3.1}$$

Ici, k^{cible} représente le nombre de lots ciblés par l'algorithme pour l'actif i, k^{prev} le nombre de lots actuellement possédés par l'utilisateur, et P_i le prix monétaire d'un lot ($100 \times$ Prix de l'action). L'opérateur $(x)^+$ signifie $\max(x, 0)$.

Chapitre 4

Approche Classique et Évaluation Dynamique

4.1 Création de la Frontière Efficiente (CVXPY)

Dans un premier temps, afin de recréer la frontière efficiente de Markowitz, j'ai utilisé la librairie `cvxpy`, un solveur d'optimisation convexe en Python.

La construction de cette frontière repose sur la résolution itérative du programme quadratique suivant. Pour une valeur de rendement cible P donnée, le problème se formule ainsi :

$$\begin{aligned} \min_{w} \quad & w^T \Sigma w \\ \text{sous les contraintes :} \quad & w^T \mu \geq P \\ & \sum_{i=1}^{N} w_i = 1 \\ & w_i \geq 0 \quad \forall i \in \{1, \ldots, N\} \end{aligned} \tag{4.1}$$

En discrétisant l'intervalle des rendements possibles et en résolvant ce problème pour chaque valeur cible P, on obtient une série de portefeuilles optimaux qui dessinent la courbe parabolique caractéristique de Markowitz.

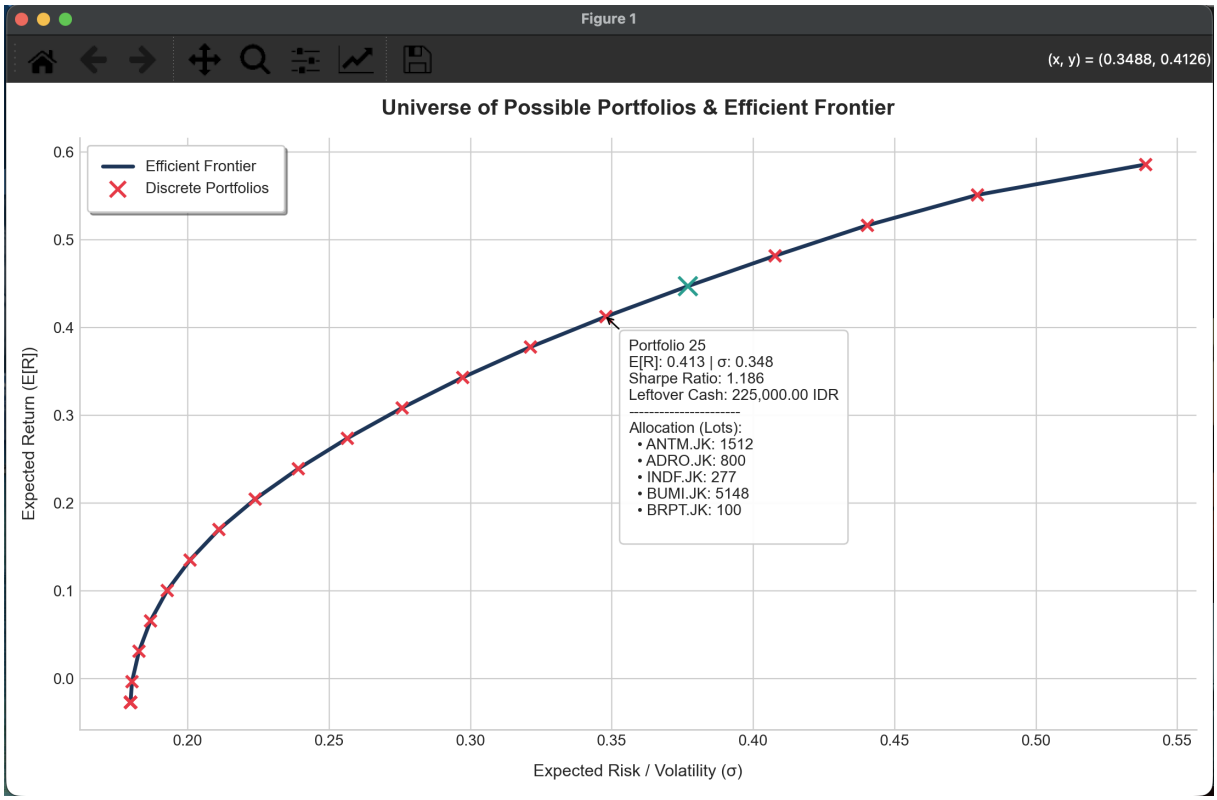


FIGURE 4.1 – L'univers des portefeuilles possibles et la Frontière Efficiente discrète.

Dans un premier temps, pour obtenir des lots, je n'avais fait que des arroundis.

La ligne bleu représente la frontière efficiente (la courbe de Pareto).

Les croix représente les portefeuilles qui sont issus de la résolution grâce à cvxpy.

Grâce au code python, cette interface est cliquable pour avoir accès à la distribution de n'importe quelle croix sur la courbe.

La croix turquoise correspond au portefeuille associé au Best Sharpe ratio. Il s'agit du portefeuille tangent maximisant $S = \frac{w^T \mu - R_f}{\sqrt{w^T \Sigma w}}$, offrant le meilleur compromis rendement/-risque.

Les 2 autres portefeuilles qui vont attirer notre regarde sera celui du minimim variance, le portefeuille le plus safe (celui tout en bas de la courbe). Et celui de maximum return, celui avec lequel on pourrait théoriquement obtenir le plus grand rendement (celui tout à droite de la courbe).

4.2 Premier Backtest : Évaluation Continue

Afin de valider ces portefeuilles théoriques, il a fallu concevoir un moteur de backtest. Le fonctionnement repose sur une fenêtre glissante (Rolling Window). L'algorithme se place au temps t_0, observe les 12 mois précédents d'historique de prix pour calculer μ et Σ, exécute le solveur CVXPY pour trouver le vecteur de poids w_{t0}, puis avance d'un mois (t_1). Au temps t_1, on calcule la performance réelle du portefeuille $w_{t0} \times$ Rendements Réels$_{t1}$, on

prélève les frais de transaction virtuels, et on ré-optimise pour le mois suivant en répétant le cycle.

Pour évaluer la surperformance, ces modèles sont comparés à un Benchmark (Stratégie Naïve 1/N). Le benchmark consiste simplement à diviser le budget en parts égales entre tous les actifs disponibles ($w_i = 1/N$), une stratégie reconnue en finance pour sa résilience naturelle face aux erreurs d'estimation des matrices de covariance. Il correspond donc au portefeuille où on laisse place au hasard.

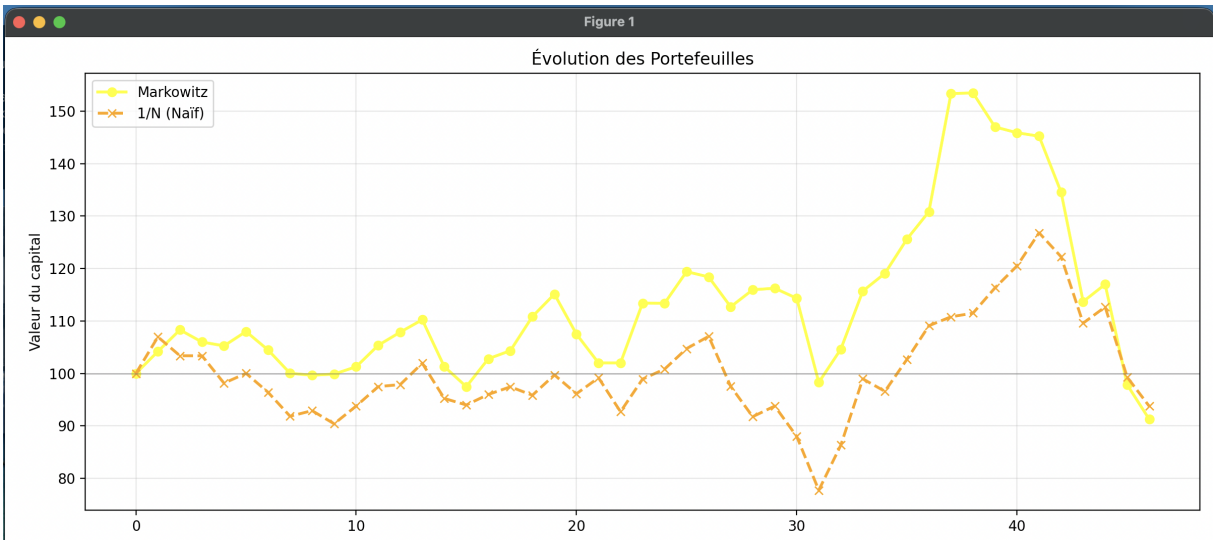


FIGURE 4.2 – Évolution de la valeur du capital : Markowitz (Best Sharpe Ratio) vs Benchmark (1/N).

La Figure 4.2 illustre les forces et les faiblesses majeures du modèle de Markowitz classique lorsqu'il est confronté aux réalités du marché. La courbe jaune représente la performance du portefeuille "Best Sharpe Ratio" rééquilibré chaque mois, tandis que la courbe orange pointillée représente la stratégie naïve 1/N.

Dans un premier temps, on constate que le portefeuille de Markowitz surperforme le benchmark lors des phases de croissance ou de stabilité (par exemple, entre les pas 15 et 30, ou lors de la forte ascension entre les pas 33 et 39). L'algorithme parvient avec succès à concentrer le capital sur les actifs présentant le meilleur historique de rendement.

Cependant, les limites de cette approche statique se révèlent lors des retournements de marché. On observe deux chutes de capital : un premier krach autour des pas 30-31, et un effondrement majeur à la fin de la simulation (après le pas 40), où le portefeuille Markowitz reperd la quasi-totalité de ses gains, finissant même légèrement en dessous du portefeuille naïf.

Cet échec s'explique par inertie temporelle du modèle : la matrice de covariance et les rendements espérés étant calculés sur une longue fenêtre glissante, l'algorithme "ne voit pas" la crise arriver. Donc l'algorithme investla plupart des fonds dans des actifs qui sont entrain de perdre de leurs valeurs.

Chapitre 5

Conception d'un Algorithme Hybride

Les résultats du backtest classique ont mis en évidence la forte inertie temporelle de Markowitz. Pour pallier ce problème critique, j'ai développé et implémenté un Algorithme Hybride.

L'objectif de cet algorithme est d'incorporer la notion de *Momentum* (la dynamique de tendance à court terme) à la théorie de Markowitz. L'algorithme hybride opère en deux phases et pondère ses décisions grâce à un facteur multiplicatif entre ces 2 phases.

5.0.1 Fonctionnement Mathématique

1. Le volet Agressif (Top-K) : L'algorithme calcule le rendement de toutes les actions sur une fenêtre très courte (ex : 3 mois). Il ne sélectionne que les K meilleures actions présentant un rendement strictement positif. Si K actions sont sélectionnées, le vecteur de poids $w_{agressif}$ leur attribue un poids de $1/K$, et 0 pour les autres. Malgré un aspect un peu simplet à premiere vue, cette algorithme permet en periode de hausse boursiere d'otbenir de très bon rendement.

2. Le volet Défensif (Markowitz Minimum Variance) : Sur une fenêtre plus longue (ex : 12 mois), l'algorithme élimine les actifs en tendance négative, puis effectue une optimisation Markowitz classique pour trouver le vecteur de poids maximisant le rendement par unité de risque, noté $w_{defensif}$.

Cela permet en période baissière de se protéger de la chute en investissant dans des actifs qui ne varient peu et donc de maintenir la valeur du portefeuille.

3. L'indicateur de Régime de Marché (α) : Pour savoir s'il faut être agressif ou défensif, l'algorithme analyse l'indice global du marché (le ^JKSE). Si le prix actuel de l'indice est très supérieur à sa moyenne historique, le marché est considéré comme "en forte tendance". Mathématiquement :

$$w_{hybride} = w_{agressif} \times (1 - \alpha) + w_{defensif} \times \alpha \tag{5.1}$$

où $\alpha \in [0, 1]$. Si le marché s'effondre (forte volatilité), $\alpha \approx 1$, et l'algorithme donne tout le poids au portefeuille défensif. Si le marché est en plein essor *momentum*, $\alpha \approx 0$, donnant la priorité aux actions Top-K.

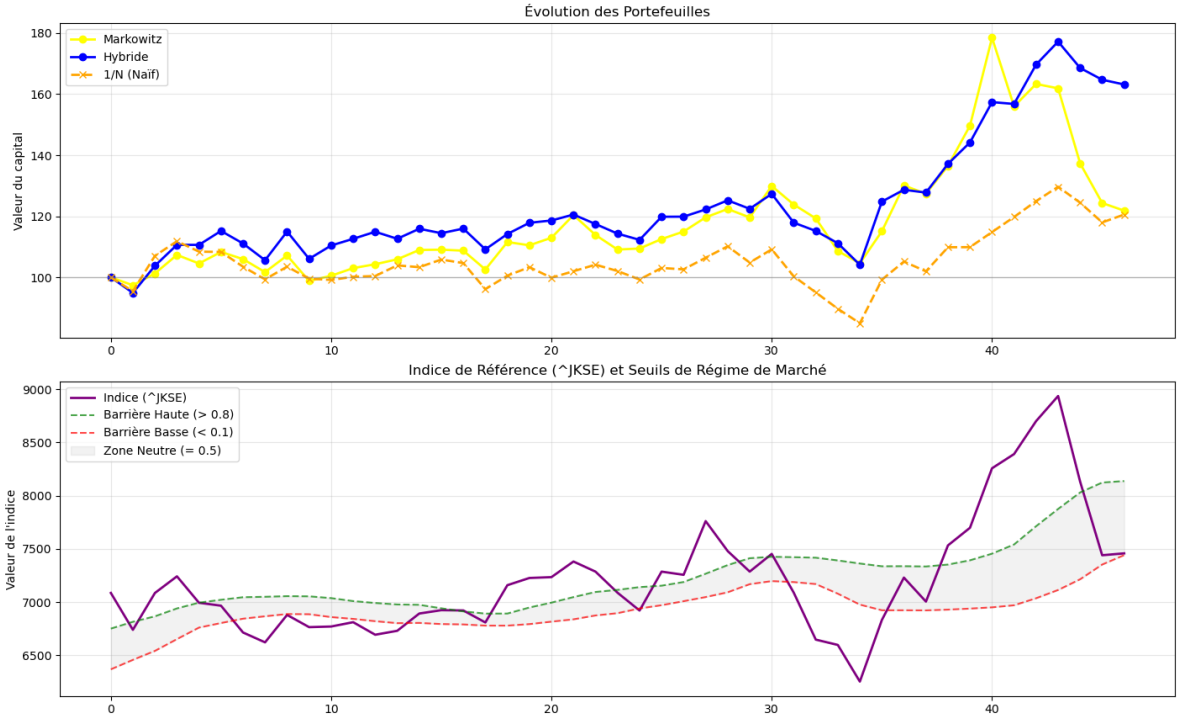


FIGURE 5.1 – Évolution des portefeuilles et détection des krachs avec l'Algorithme Hybride.

Les résultats graphiques (Figure 5.1) illustrent l'efficacité de cet indicateur de régime. La partie inférieure du graphique montre l'évolution de l'indice ^JKSE (courbe violette) face à ses barrières de régime. Entre les pas de temps 30 et 35, l'indice subit une chute abrupte, plongeant sous sa barrière basse (ligne rouge pointillée).

En observant simultanément la partie supérieure du graphique, on constate les faiblesses du portefeuille Markowitz classique "Best Sharpe Ratio" (courbe jaune) : figé sur des moyennes historiques, il ne réagit pas à la la baisse du court des actifs et subit une baisse de capital. À l'inverse, l'Algorithme Hybride (courbe bleue) détecte immédiatement ce changement de régime et se rétracte vers une allocation défensive. Ainsi, au moment précis où le marché chute, la valeur du portefeuille hybride stagne et résiste au krach au lieu de s'effondrer comme le fait Markowitz. Lorsque l'indice repart finalement à la hausse (après le pas 35), l'algorithme hybride repasse en mode agressif, capture le momentum haussier, et termine avec une performance globale nettement supérieure à Markowitz et au Benchmark Naïf (courbe orange).

Chapitre 6

De l'Algorithme Génétique à MOSMA

6.1 Nécessité de l'optimisation discrète et Algorithme Génétique

Malgré le succès théorique de l'approche hybride, un problème concret persistait pour l'application finale : les poids obtenus étaient toujours continus. Sur le marché indonésien, un utilisateur avec 10 000 000 Rp ne peut pas acheter 12.3% d'une action dont le lot coûte 1 500 000 Rp.

Dans un premier temps, j'ai développé un Algorithme Génétique (GA). Au lieu de chercher des pourcentages, la population initiale de l'algorithme était composée de vecteurs d'entiers représentant directement des quantités de lots. Les fonctions de croisement et de mutation modifiaient aléatoirement ces lots, avec une fonction de *fitness* pénalisant massivement les individus dépassant le budget ou s'en éloignant trop.

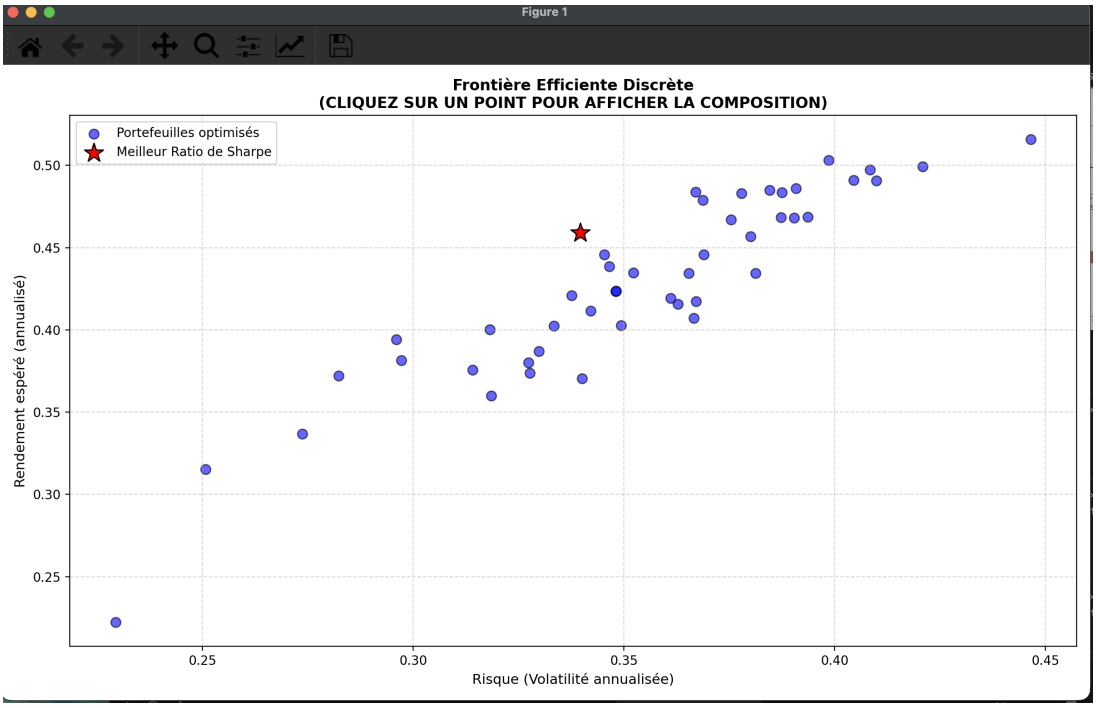


FIGURE 6.1 – Frontière générée par l'Algorithme Génétique : un amas de points irrégulier et sous-optimal.

Comme l'illustre la Figure, la frontière générée par l'algorithme génétique souffre d'un manque d'homogénéité (amas de points irréguliers, zones vides). De plus, l'introduction d'une barrière stricte limitant le poids maximum par action à environ 55% complexifie considérablement l'espace de recherche combinatoire. En conséquence, l'algorithme génétique s'est avéré très lent, nécessitant près de 3 minutes de temps de calcul pour converger vers ce résultat, ce qui est donc améliorable.

6.2 Implémentation algorithmique de MOSMA (Logique NSGA-II)

Face aux limites de temps de calcul du génétique classique, j'ai implémenté l'algorithme MOSMA (*Multi-Objective Slime Mould Algorithm*). Cet algorithme s'attaque directement à un problème à double objectif : maximiser le rendement espéré tout en minimisant le risque.

À chaque itération, l'algorithme évalue la population en termes de Risque et de Rendement espéré. Il applique ensuite la fonction `tri_non_domine`, qui repose sur la logique de NSGA-II. L'algorithme vérifie si chaque portefeuille est dominé par un autre (un portefeuille est dominé s'il offre moins de rendement pour plus de risque qu'un autre point). Les portefeuilles non dominés constituent le Front de Pareto de l'itération actuelle. Ensuite, l'algorithme attribue les poids biologiques W du *Slime Mould* pour orienter la convergence

des individus vers les meilleures solutions, tout en appliquant une vibration stochastique `vb` pour maintenir la diversité.

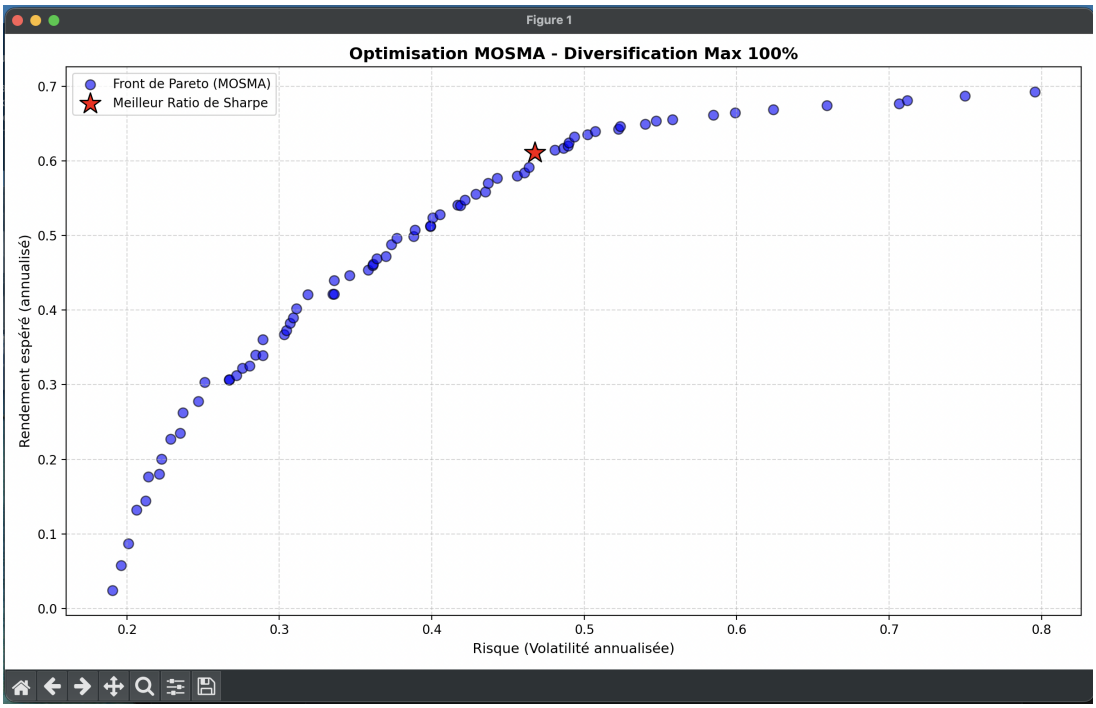


FIGURE 6.2 – Frontière générée par MOSMA (logique NSGA-II) avec une nette amélioration de la convergence.

La Figure démontre l'efficacité de MOSMA face à l'algorithme génétique classique. La courbe de Pareto est beaucoup mieux définie et plus proche de l'optimum. Surtout, l'algorithme n'a mis que 10 secondes à s'exécuter (contre 3 minutes pour le génétique).

Toutefois, en observant attentivement la Figure 6.2, on remarque que la distribution des portefeuilles le long de la courbe n'est pas parfaitement régulière (certains points sont très rapprochés, d'autres plus espacés). Ce phénomène est une limite connue de la méthode NSGA-II qui utilise la distance de congestion (Crowding Distance) : bien qu'elle favorise la diversité globale, elle ne garantit pas une équidistance mathématique parfaite des solutions sur le front.

6.3 Uniformisation via NSGA-III

Pour s'assurer que les solutions proposées à l'utilisateur couvrent équitablement tout le spectre du risque, j'ai couplé MOSMA avec la méthode de tri NSGA-III. La fonction filtrer_archive_nsga3 est appelée à la toute fin de l'algorithme MOSMA. Elle définit 100 lignes de référence parfaitement équidistantes dans l'espace 2D (Risque-Rendement) normalisé. Elle parcourt l'énorme archive générée par MOSMA, calcule la distance orthogonale de chaque portefeuille par rapport à ces vecteurs, et ne conserve que l'individu le plus proche de chaque ligne.

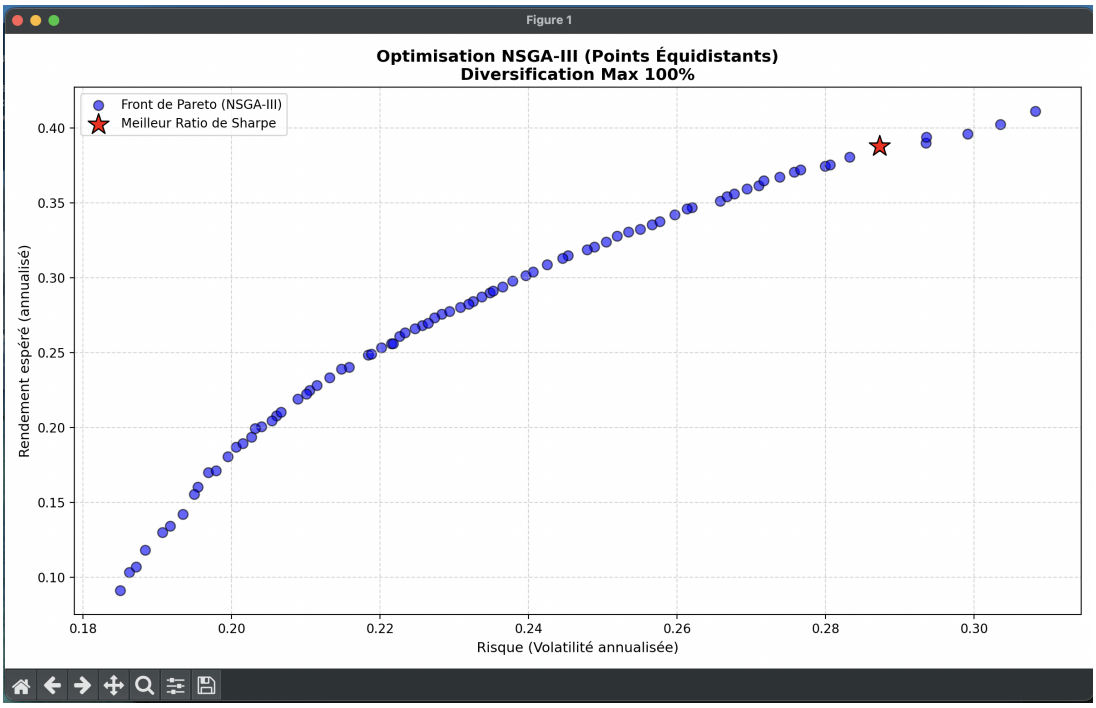


FIGURE 6.3 – Uniformisation absolue du Front de Pareto grâce au filtre NSGA-III.

La Figure illustre le résultat final. Malgré la présence de contraintes strictes (budget, achat par lots entiers, et barrière maximale par action limitant l'espace de recherche), le filtre NSGA-III réussit à sculpter une courbe où la distance entre chaque niveau de risque est beaucoup plus homogène que notre résultat précédent.

Chapitre 7

Création de la Plateforme Web (Streamlit)

La concrétisation de l'ensemble de ces recherches algorithmiques est le déploiement d'une application web interactive, développée via le framework Streamlit. Cette plateforme est conçue pour être hébergée et mise à disposition par le département P2MS de l'ITB. L'objectif principal de l'interface est de masquer la complexité mathématique des algorithmes métaheuristiques (MOSMA, NSGA-III) pour offrir aux investisseurs un outil d'aide à la décision intuitif et rapide.

7.1 Paramétrage de l'Investissement

Le panneau de contrôle latéral permet à l'utilisateur de configurer les variables fondamentales de son investissement, qui seront injectées comme contraintes dans le solveur MOSMA :

- Liquidités disponibles (Available Cash) : Fixé par défaut à 10 millions de Roupies Indonésiennes (Rp) dans notre exemple. C'est la contrainte budgétaire stricte. Frais de courtage (Brokerage Fees) : L'utilisateur peut modéliser précisément l'asymétrie de son propre courtier (ex : 0.15% à l'achat, 0.25% à la vente).
- Poids maximum par action : Un curseur de contrôle du risque permettant de forcer la diversification (ex : limiter l'exposition à 15% maximum par titre).
- Sélection des actifs et Portefeuille existant : L'utilisatueur va sélectionner les stocks dans lesquels ils souhaitent investir et peut en rajouter en les écrivants dans la section add manual. l'outil permet de renseigner un portefeuille actuel (dans le cas où l'utilsateur possède dejâ des stocks) : l'algorithme calculera alors les frais de transaction uniquement sur le rééquilibrage nécessaire, simulant un rebalancing sans forcer la liquidation totale.

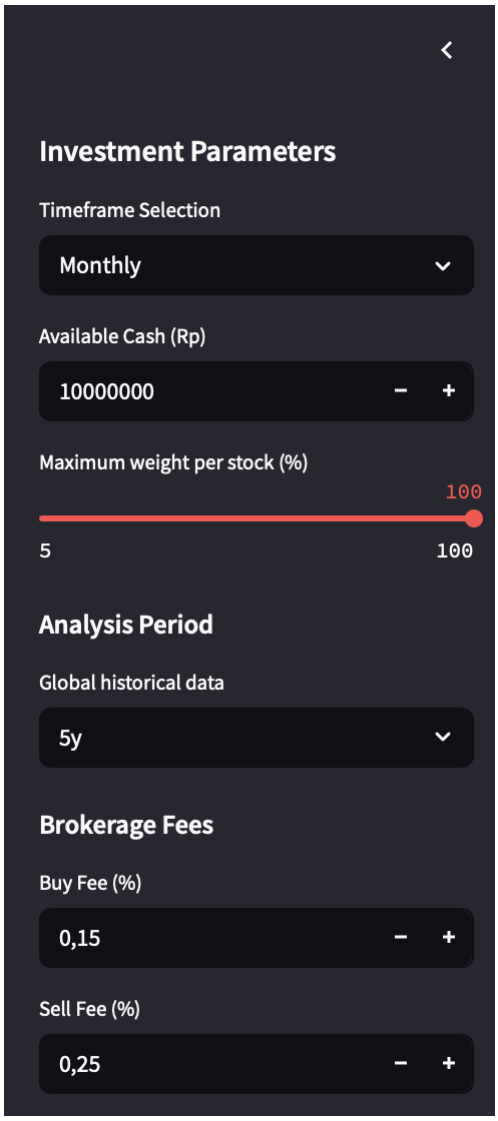


FIGURE 7.1 – Paramètres financiers globaux.

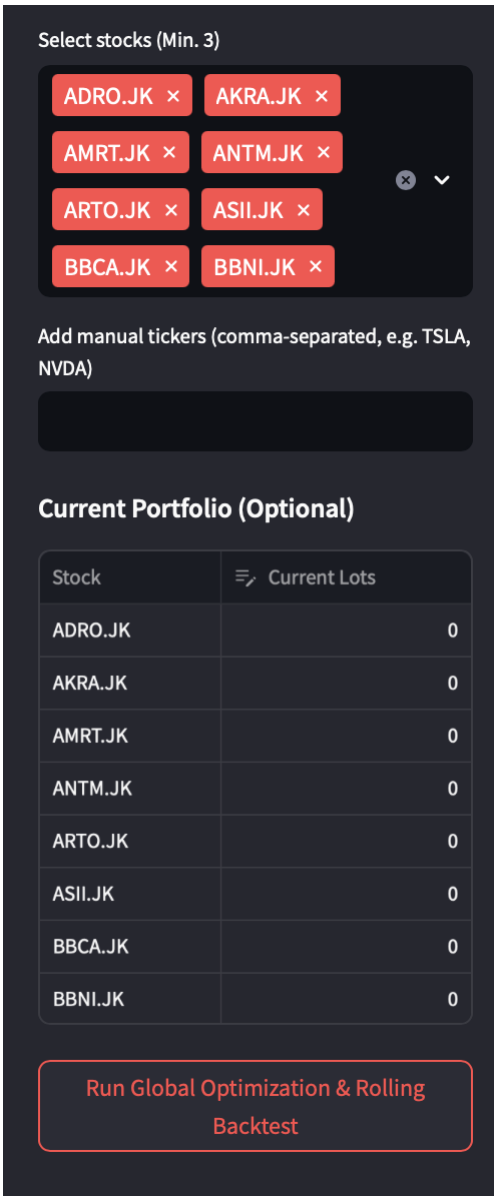


FIGURE 7.2 – Sélection des actifs et portefeuille actuel.

7.2 Visualisation et Sélection sur le Front de Pareto

Une fois l'optimisation lancée, l'application génère dynamiquement le Front de Pareto. Contrairement aux approches continues classiques, chaque point de cette courbe représente un portefeuille strictement réalisable en lots entiers. Le graphique interactif utilise un gradient de couleur pour mettre en évidence le Ratio de Sharpe (jaune pour les ratios élevés, bleu foncé pour les ratios faibles).

Lorsqu'un utilisateur clique sur un point de la courbe, le tableau de droite se met à jour pour afficher le Plan d'Exécution associé. Dans l'exemple sélectionné (marqué par une étoile), pour atteindre un rendement net espéré de 33.0% avec un risque de 28.5%,

l'outil génère l'ordre direct à transmettre au courtier : « Acheter 22 lots de ADRO.JK (58.8%) et 11 lots de BBNI.JK (41.2%) ».



FIGURE 7.3 – Interface du Front de Pareto interactif et du plan d'exécution ciblé.

7.3 Allocation Hybride et Détection de Régime

Au-delà de la sélection manuelle, la plateforme propose automatiquement la composition du Portefeuille Hybride. Comme détaillé au Chapitre 5, cette allocation combine de manière dynamique le portefeuille le plus agressif (Max Return) et le plus défensif (Min Risk) de la courbe de Pareto.



FIGURE 7.4 – Tableau de l'Allocation Hybride avec indicateur de Régime de Marché (Alpha).

Dans la capture d'écran, l'indicateur de régime de marché (α) affiche une valeur de 0.10. Cela signifie que l'indice de marché global est détecté en tendance fortement haussière (très au-dessus de sa moyenne). Le moteur d'hybridation choisit donc d'allouer 90% du capital à la stratégie agressive Max Return, et seulement 10% à la protection Min Risk. Le tableau final reflète cette agressivité avec une allocation massive sur ADRO.JK (92% du portefeuille cible).

7.4 Analyse Comparative par Backtest (Validation Out-of-Sample)

La dernière fonctionnalité majeure du site est le simulateur de Rolling Window Backtest. Cet outil permet à l'utilisateur de valider visuellement et statistiquement la robustesse de l'approche MOSMA sur les données historiques avant d'engager son capital.

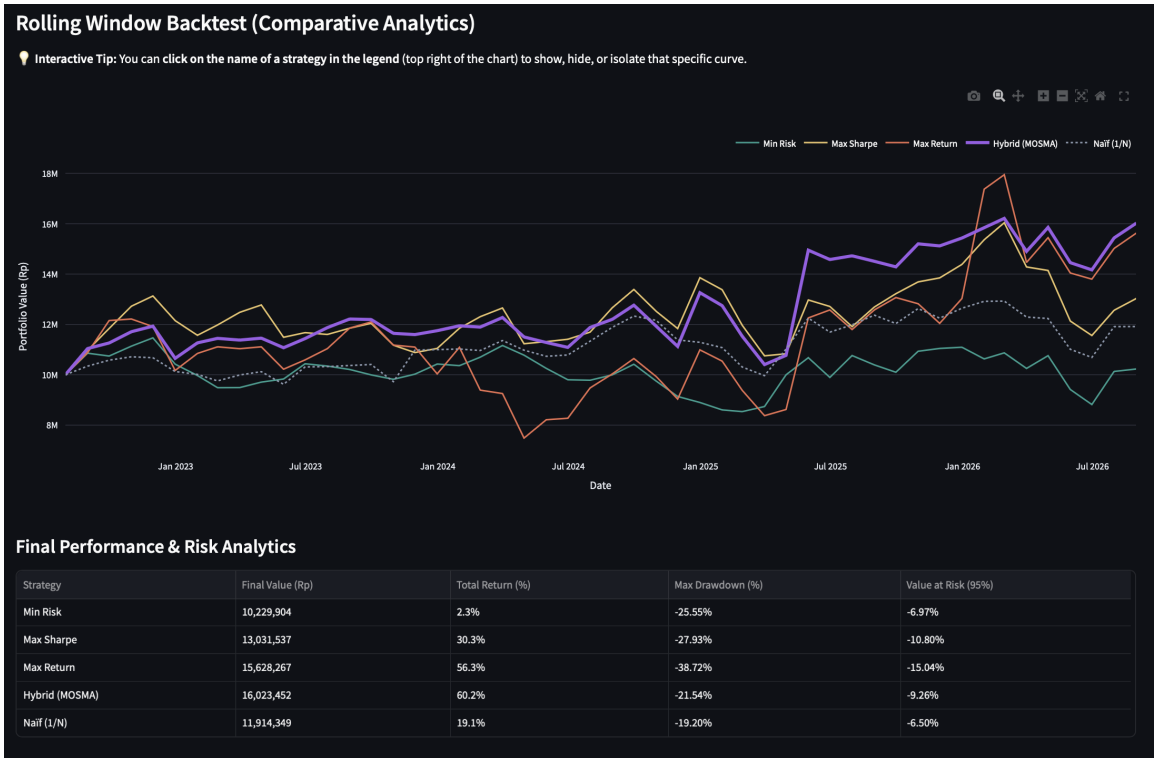


FIGURE 7.5 – Évolution historique des différentes stratégies et métriques de risque final.

L'analyse du tableau de performances finales révèle des résultats probants :

— La stratégie Naïve (1/N), bien que résiliente, affiche un rendement modeste de 19.1% avec un Drawdown (chute maximale du capital) de -19.20%.

— La stratégie Max Sharpe améliore logiquement ce score avec 30.3% de rendement, mais au prix d'une volatilité subie plus forte (-27.93% de Max Drawdown).

— La stratégie Hybride (MOSMA) surclasse toutes les autres en atteignant un rendement total de 60.2%. Sa gestion dynamique du risque via le facteur α lui permet de restreindre son Max Drawdown à -21.54%, protégeant mieux le capital en cas de krach que les stratégies Max Sharpe ou Max Return.

Donc ce tableau permet à l'utilisateur de savoir comment investir tout en ayant accès à des métriques pour avoir une idée de ce qui aurait pu se passer s'il avait commencé à investir un an plus tôt.

Chapitre 8

Tentative de Prédiction Boursière via LSTM

Dans l'optique d'améliorer l'allocation du portefeuille en anticipant le marché plutôt qu'en réagissant uniquement à l'historique, une étape de recherche s'est concentrée sur la prédiction temporelle via un modèle Deep Learning. L'objectif était de prédire le prix de l'action (ADRO.JK) pour le lendemain, puis sur un horizon étendu d'une semaine (7 jours).

8.1 Préparation des données et Modélisation

Les premières tentatives consistant à prédire directement la valeur absolue du prix de l'action se sont révélées infructueuses. Les séries temporelles financières étant fortement non-stationnaires, le modèle avait tendance à se contenter de renvoyer le prix de la veille pour minimiser son erreur, ce qui le rendait inutile pour l'anticipation.

Pour corriger ce phénomène, l'architecture a été modifiée pour prédire le taux d'accroissement quotidien plutôt que le prix brut. Les données ont été mises à l'échelle à l'aide d'un MinMaxScaler entre 0 et 1, et structurées sous forme de séquences glissantes de 30 jours.

L'architecture retenue est composée de deux couches LSTM superposées de 50 neurones chacune, séparées par des couches de Dropout (20%) pour limiter le surapprentissage. Le modèle se termine par des couches denses pour restituer une prédiction unique, optimisée par l'algorithme adam via la fonction de perte MAE (Mean Absolute Error).

8.2 Analyse de la Prédiction à 1 Jour (Fenêtre Glissante Réelle)

La première évaluation consistait à simuler la prédiction du lendemain sur les 60 derniers jours de l'historique[cite : 20]. Chaque jour t, l'algorithme reçoit la "vraie" séquence des 30 jours précédents (de $t-30$ à $t-1$) pour prédire la variation au temps t[cite : 20].

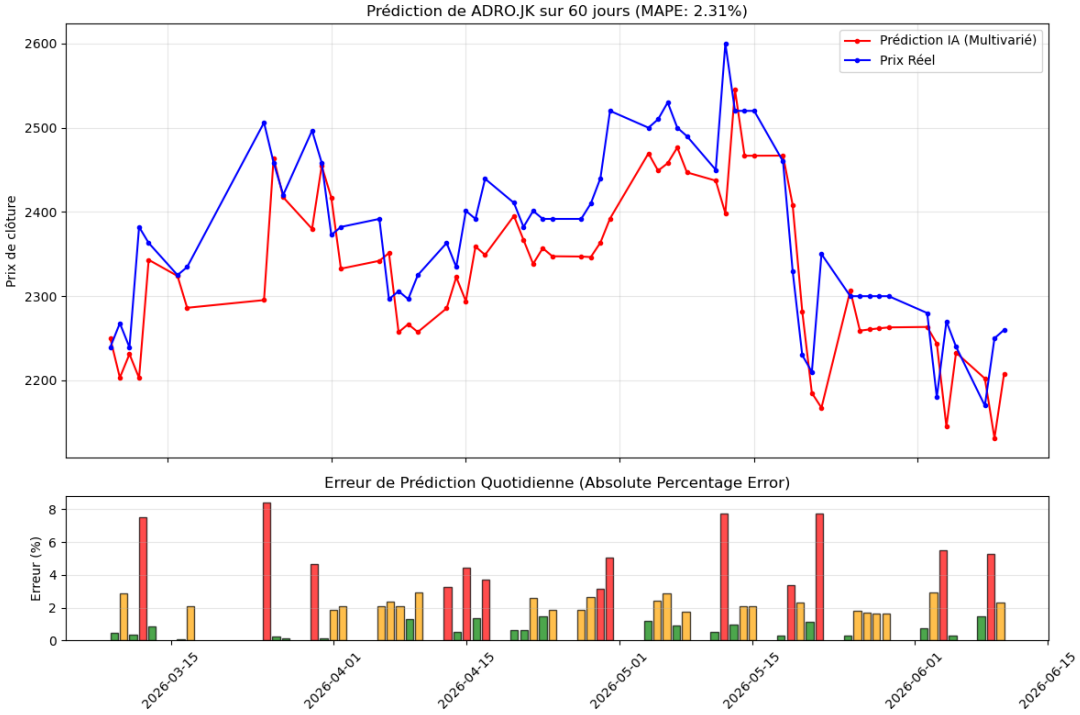


FIGURE 8.1 – Prédiction jour par jour de ADRO.JK sur 60 jours.

Les résultats graphiques illustrent un MAPE de seulement 2.31%. Bien que cette erreur semble faible, l'analyse visuelle de la courbe révèle une limite intrinsèque des LSTM univariés sur des marchés volatils : la courbe prédite (en rouge) tend à suivre la courbe réelle (en bleu) avec un léger décalage temporel. Le réseau capture correctement la tendance moyenne et l'ordre de grandeur de la variation, mais il échoue à anticiper les chocs brusques (les pics et les creux inversés).

8.3 Analyse de la Prédiction Récursive à 7 Jours

Pour pousser le modèle plus loin, une prédiction récursive a été implémentée pour un horizon de 7 jours. Contrairement à l'approche précédente, l'algorithme ne reçoit plus de données réelles après le jour t_0[cite : 20]. Pour prédire le jour $t+2$, il est forcé d'utiliser sa propre prédiction du jour $t+1$ qu'il réinjecte dans sa fenêtre de séquence[cite : 20]. L'Intelligence Artificielle "se nourrit d'elle-même".

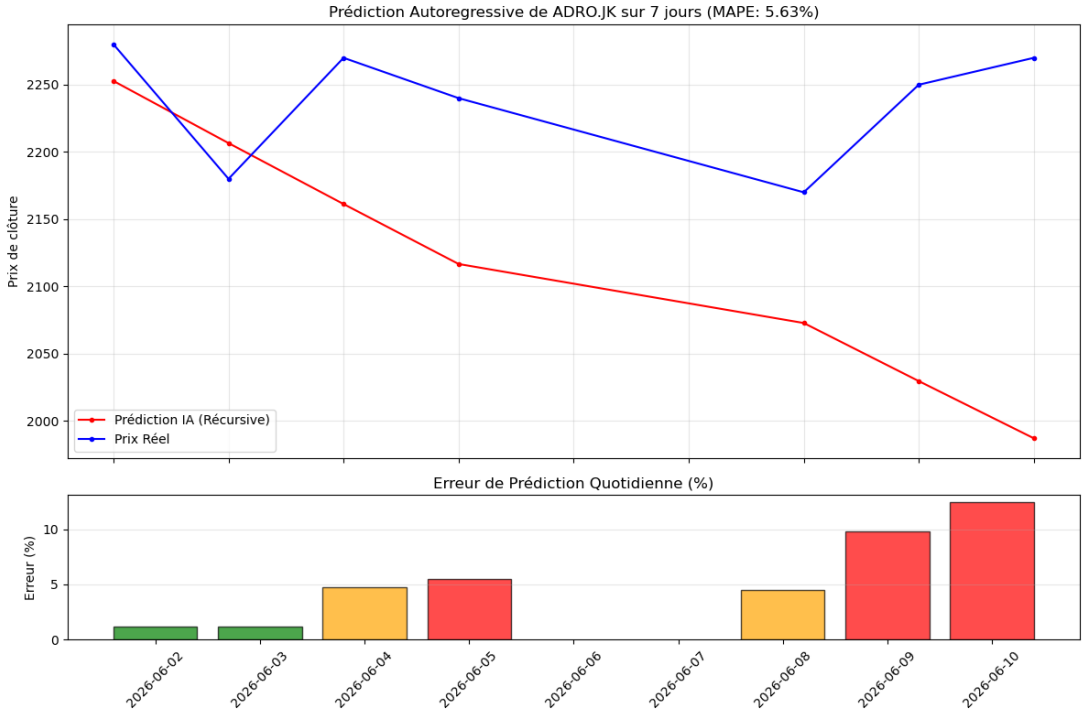


FIGURE 8.2 – Prédiction Autorégressive (Récursive) de ADRO.JK sur 7 jours.

Cette tentative s'est révélée non concluante. Comme le montre la Figure , la prédiction (en rouge) trace une ligne descendante générique, tandis que le prix réel du marché (en bleu) subit de fortes fluctuations aléatoires (chute, rebond, chute). Sans la correction apportée par les données réelles quotidiennes, les erreurs d'estimation du modèle s'accumulent et s'amplifient de jour en jour. Le MAPE grimpe à 5.63%, et les barres d'erreur quotidiennes explosent au-delà de 10% vers la fin de la semaine.

8.4 Conclusion sur l'utilisation du LSTM

Cette série d'expériences démontre que l'utilisation d'un simple réseau LSTM ne suffira pas pour réussir à capter les variations aléatoires du marché indonésien.

C'est pour cette raison que je n'ai pas incorporé ces prédictions dans le site web.

Chapitre 9

Conclusion et Perspectives

9.1 Bilan de l'Étude

L'objectif global consistant à fournir un outil d'optimisation robuste, rapide et adapté aux contraintes réelles du marché indonésien a été atteint. En remplaçant les fondations mathématiques continues par des approches métaheuristiques hybrides (MOSMA exploité avec les principes de NSGA-III), le système génère des ordres d'achat en termes de lots entiers et en intégrand les couts d'achats et de vente.

9.2 Le Biais du Survivant : Une Limite Majeure

Il est impératif de souligner que les rendements spectaculaires affichés dans les tests hors-échantillon (ex : plus de 60% de rendement total sur l'approche Hybride) sont largement biaisés par un phénomène statistique bien connu en finance : le biais du survivant.

Dans notre méthodologie, nous avons sélectionné l'univers d'investissement actuel (les entreprises présentes dans l'indice IDX30 au moment de l'étude) et nous avons téléchargé leur historique de prix sur les 5 dernières années pour effectuer le backtest. Cependant, cela signifie que nous n'avons testé nos stratégies que sur des entreprises qui ont « survécu » et réussi à se hisser ou à se maintenir dans l'IDX30 jusqu'à aujourd'hui.

Nous ignorons volontairement toutes les entreprises qui faisaient partie de cet indice il y a 3 ou 4 ans, mais qui ont depuis fait faillite, subi des krachs majeurs ou été radiées de la cote. En conséquence, l'algorithme de backtest ne peut choisir qu'entre des actions globalement "gagnantes" sur la période étudiée, ce qui gonfle artificiellement les performances simulées. Pour qu'une future itération de cet outil soit académiquement correcte, il faudra utiliser une base de données contenant aussi les entreprises supprimées de l'IDX30.

9.3 Perspectives Futures

Outre la correction du biais du survivant pour les simulations futures, plusieurs axes d'amélioration peuvent être envisagés :

— Intégration de données macro-économiques (inflation, taux directeurs de la Bank Indonesia) au sein des variables du modèle de Machine Learning pour tenter d'obtenir des résultats plus probants.

— Créer une architecture de cache dans le site web pour réduire le temps de calcul au cours de son utilisation.

Bibliographie

[1] Markowitz, H. (1952). Portfolio Selection. *The Journal of Finance*, 7(1), 77-91.

[2] Li, S., Chen, H., Wang, M., Heidari, A. A., & Mirjalili, S. (2020). Slime mould algorithm : A new method for stochastic optimization. *Future Generation Computer Systems*, 111, 300-323.

[3] Deb, K., & Jain, H. (2014). An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point Based Nondominated Sorting Approach, Part I : Solving Problems with Box Constraints. *IEEE Transactions on Evolutionary Computation*, 18(4), 577-601.

[4] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.

Annexe A

Codes Sources Python Développés

A.1 Prédiction LSTM (`ml_stage.py`)

```
import yfinance as yf
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout

# Configuration
MOIS_MOMENTUM = 6
MOIS_HRP = 12
MOIS_MAX = max(MOIS_MOMENTUM, MOIS_HRP)
MOIS_TEST = 36
WINDOW_SIZE = 30

# Liste des actifs
assets = ["ADRO.JK", "AKRA.JK", "AMRT.JK", "ANTM.JK", "ARTO.JK", "ASII.JK",
          "BBCA.JK", "BBNI.JK", "BBRI.JK", "BMRI.JK", "BRPT.JK", "BUMI.JK",
          "CPIN.JK", "EMTK.JK", "ESSA.JK", "GOTO.JK", "HRUM.JK", "ICBP.JK",
          "INDF.JK", "INKP.JK", "ISAT.JK", "KLBF.JK", "MDKA.JK", "MEDC.JK",
          "PGAS.JK", "PTBA.JK", "TLKM.JK", "TOWR.JK", "UNTR.JK", "UNVR.JK"]

data = yf.download(assets, period="5y")['Close']
data = data.dropna(axis=1, how='all')
data=data.ffill().bfill()
scaler=MinMaxScaler(feature_range=(0,1))
```

```
27
28 def create_sequences(data,window_sizes,target_idx):
29     X=[]
30     y=[]
31     for i in range(len(data)-window_sizes):
32         X.append(data[i:i+window_sizes])
33         y.append(data[i+window_sizes,target_idx])
34     return np.array(X),np.array(y)
35
36 data['Variation']=data['ADRO.JK'].pct_change()
37 data=data.dropna()
38 data_scaled=pd.DataFrame(scaler.fit_transform(data),columns=data.columns
       ,index=data.index)
39 idx=data_scaled.columns.get_loc('Variation')
40 X,y=create_sequences(data_scaled.values,WINDOW_SIZE,idx)
41
42 training_data_len = int(len(X) * 0.8)
43 X_train, y_train = X[:training_data_len], y[:training_data_len]
44 X_test, y_test = X[training_data_len:], y[training_data_len:]
45
46 model = Sequential()
47 model.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.
       shape[1], X_train.shape[2])))
48 model.add(Dropout(0.2))
49 model.add(LSTM(units=50, return_sequences=False))
50 model.add(Dropout(0.2))
51 model.add(Dense(units=25))
52 model.add(Dense(units=1))
53 model.compile(optimizer='adam', loss='mean_absolute_error')
54
55 model.fit(X_train, y_train, batch_size=32, epochs=20)
```

A.2 Application Streamlit (app.py)

```
1 import streamlit as st
2 import numpy as np
3 import pandas as pd
4 import plotly.graph_objects as go
5 import yfinance as yf
6 from utils import load_data
7 from strategies import execute_mosma
8 from backtest_engine import build_portfolio_candidates,
       rolling_mosma_backtest, regime_switch
9
10 st.set_page_config(page_title="Optimiseur IDX", layout="wide")
11 st.title("Optimiseur de Portefeuille avec Frais Reels (MOSMA)")
```

```
12
13 # Parametres de la barre laterale
14 TOTAL_BUDGET = st.sidebar.number_input("Liquidites Disponibles (Rp)",
       min_value=0, value=0, step=10000000)
15 MAX_WEIGHT_PER_STOCK = st.sidebar.slider("Poids maximum par action (%)",
        min_value=5, max_value=100, value=100, step=5) / 100.0
16
17 BUY_FEE = st.sidebar.number_input("Frais d'Achat (%)", value=0.15) /
       100.0
18 SELL_FEE = st.sidebar.number_input("Frais de Vente (%)", value=0.25) /
       100.0
```

A.3 Moteur de Backtest (backtest_engine.py)

```
1  import numpy as np
2  import pandas as pd
3  from utils import calculate_transaction_cost
4  from strategies import execute_mosma
5
6  RISK_FREE_RATE = 0.05
7
8  def regime_switch(indice_train):
9      if isinstance(indice_train, pd.DataFrame):
10         indice_train = indice_train.squeeze()
11
12     if len(indice_train) == 0:
13         return 0.5
14
15     barriere = float(indice_train.std() * 0.5)
16     current_price = float(indice_train.iloc[-1])
17     average = float(indice_train.mean())
18
19     if current_price > average + barriere:
20         return 0.8
21     elif current_price < average - barriere:
22         return 0.1
23     else:
24         return 0.5
25
26 def exact_oos_step(wealth, w, k_prev, prices_start, prices_end, buy_fee,
       sell_fee):
27     w = np.nan_to_num(w)
28     w = np.clip(w, 0, 1)
29     if np.sum(w) > 0:
30         w = w / np.sum(w)
31
```

```
    target_shares = np.floor((w * wealth) / prices_start).astype(int)
    target_lots = target_shares // 100

    tc = calculate_transaction_cost(k_prev, target_lots, prices_start,
    buy_fee, sell_fee)

    invested_value = np.sum(target_lots * 100 * prices_start)
    cash = wealth - invested_value - tc

    new_invested_value = np.sum(target_lots * 100 * prices_end)
    new_wealth = new_invested_value + cash

    return new_wealth, target_lots
```

A.4 Stratégies Multi-Objectifs (strategies.py)

```
import numpy as np
from utils import calculate_transaction_cost

def repair_weights(weights, n_assets, weight_limit):
    weights = np.clip(weights, 0, None)
    if np.sum(weights) == 0:
        return np.ones(n_assets) / n_assets
    effective_limit = max(weight_limit, 1.0 / n_assets)
    weights = weights / np.sum(weights)
    capped = np.zeros(n_assets, dtype=bool)

    for _ in range(n_assets):
        over = weights > effective_limit
        if not np.any(over):
            break
        weights[over] = effective_limit
        capped |= over
        free = ~capped
        if not np.any(free):
            break
        remaining = 1.0 - np.sum(weights[capped])
        free_sum = np.sum(weights[free])

        if free_sum <= 1e-12:
            weights[free] = remaining / np.sum(free)
        else:
            weights[free] = weights[free] / free_sum * remaining

    weights = weights / np.sum(weights)
    return weights
```

```
31
32 def evaluate_objectives(weights, expected_returns, cov_matrix,
   current_prices, k_prev, total_wealth, alpha_buy, alpha_sell):
33     ret = np.dot(expected_returns, weights)
34     risk = np.sqrt(np.dot(weights, np.dot(cov_matrix, weights)))
35     target_shares = np.floor((weights * total_wealth) / current_prices).
   astype(int)
36     target_lots = target_shares // 100
37     tc = calculate_transaction_cost(k_prev, target_lots, current_prices,
    alpha_buy, alpha_sell)
38     tc_pct = tc / (total_wealth + 1e-9)
39     net_ret = ret - tc_pct
40     return np.array([-net_ret, risk])
41
42 def tri_non_domine(fitness_pop):
43     taille_pop = fitness_pop.shape[0]
44     domine_par = [[] for _ in range(taille_pop)]
45     compteur_domination = np.zeros(taille_pop)
46     fronts = [[]]
47
48     for i in range(taille_pop):
49         for j in range(taille_pop):
50             if (fitness_pop[i, 0] <= fitness_pop[j, 0] and fitness_pop[i
   , 1] <= fitness_pop[j, 1]) and \
51                 (fitness_pop[i, 0] < fitness_pop[j, 0] or fitness_pop[i,
   1] < fitness_pop[j, 1]):
52                 domine_par[i].append(j)
53             elif (fitness_pop[j, 0] <= fitness_pop[i, 0] and fitness_pop
   [j, 1] <= fitness_pop[i, 1]) and \
54                  (fitness_pop[j, 0] < fitness_pop[i, 0] or fitness_pop[j
   , 1] < fitness_pop[i, 1]):
55                 compteur_domination[i] += 1
56
57         if compteur_domination[i] == 0:
58             fronts[0].append(i)
59     return fronts[0]
60
61 def filtrer_archive_nsga3(archive_poids, archive_fitness, num_ref_points
   =50):
62     if len(archive_fitness) <= 1:
63         return archive_poids, archive_fitness
64
65     fitness_arr = np.array(archive_fitness)
66     z_min = np.min(fitness_arr, axis=0)
67     z_max = np.max(fitness_arr, axis=0)
68     denom = np.where(z_max - z_min == 0, 1e-9, z_max - z_min)
69     fitness_norm = (fitness_arr - z_min) / denom
```

```
70
71      ref_points = np.array([[1 - i/(num_ref_points-1), i/(num_ref_points
        -1)]
72                             for i in range(num_ref_points)])
73      normes_ref = np.linalg.norm(ref_points, axis=1)
74
75      poids_sel = []
76      fit_sel = []
77      vus = set()
78
79      for i, ref in enumerate(ref_points):
80          ref_dir = ref / normes_ref[i]
81          proj = np.outer(np.dot(fitness_norm, ref_dir), ref_dir)
82          dist = np.linalg.norm(fitness_norm - proj, axis=1)
83          best_idx = np.argmin(dist)
84
85          cle = tuple(np.round(archive_fitness[best_idx], 6))
86          if cle not in vus:
87              vus.add(cle)
88              poids_sel.append(archive_poids[best_idx])
89              fit_sel.append(archive_fitness[best_idx])
90
91      return poids_sel, fit_sel
```

A.5 Fonctions Utilitaires (utils.py)

```
1  import yfinance as yf
2  import numpy as np
3
4  def load_data(assets, period):
5      data = yf.download(assets, period=period)['Close']
6      data = data.dropna(axis=1, how='all').ffill().dropna()
7      return data
8
9  def calculate_transaction_cost(k_prev, k_target, current_prices,
       alpha_buy, alpha_sell):
10     if k_prev is None or np.sum(k_prev) == 0:
11         return alpha_buy * np.sum(100 * current_prices * k_target)
12     else:
13         lots_bought = np.maximum(k_target - k_prev, 0)
14         lots_sold = np.maximum(k_prev - k_target, 0)
15
16         tc_buy = alpha_buy * np.sum(100 * current_prices * lots_bought)
17         tc_sell = alpha_sell * np.sum(100 * current_prices * lots_sold)
18
19         return tc_buy + tc_sell
```